

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ I



BÁO CÁO
HỆ ĐIỀU HÀNH NHÚNG

ĐỀ TÀI: HỆ THỐNG ĐÈN THÔNG MINH

Lớp: DTMT01

Nhóm học phần: 02

Giảng viên hướng dẫn: Lương Công Duẩn

Sinh viên thực hiện:

Nguyễn Hữu Sơn MSV B22DCDT270

Lương Công Thắng MSV B22DCDT306

Trịnh Quang Huy MSV B22DCDT142

Đỗ Đức Thuận MSV B22DCDT310

HÀ NỘI, T4/2026

MỤC LỤC

1. TỔNG QUAN HỆ THỐNG.....	4
1.1. Mục tiêu thiết kế	4
1.2. Thành phần phần cứng.....	4
1.3. Sơ đồ kiến trúc tổng thể	5
2. KIẾN TRÚC PHẦN MỀM.....	6
2.1. Cấu trúc dữ liệu dùng chung — SystemState.....	6
2.2. Mô hình đa luồng (Multi-threading)	6
3.1. Driver BH1750 — Cảm biến ánh sáng môi trường.....	7
3.2. Driver DS3231 — Đồng hồ thời gian thực (RTC).....	7
3.3. Driver PIR — Cảm biến chuyển động	8
3.4. Driver PWM — Điều khiển LED.....	8
3.5. Driver SH1106 — Màn hình OLED 128×64	9
4. ỨNG DỤNG USERSPACE	10
4.1. Thread 1 — Đọc cảm biến.....	10
4.2. Thread 2 — Logic điều khiển (Control Thread, chu kỳ 20ms).....	10
4.2.1. Thuật toán quyết định độ sáng mục tiêu.....	10
4.2.2. Hiệu ứng Fade PWM.....	11
4.2.3. Ghi nhật ký năng lượng	11
4.3. Thread 3 — Web Server	11
4.4. Thread 4 — Hiển thị OLED (chu kỳ 1 giây).....	11
5. DEVICE TREE (DTS).....	12
5.1. Cấu hình Pin Mux	12
5.2. Node thiết bị.....	12
6. QUY TRÌNH KHỞI ĐỘNG VÀ DỪNG HỆ THỐNG.....	13
6.1. Trình tự khởi động tự động — S99smartlamp.....	13
6.2. Trình tự dừng hệ thống (SIGINT/SIGTERM).....	13
7. GIAO DIỆN WEB DASHBOARD	14
7.1. Các thành phần giao diện	14
7.2. Cơ chế cập nhật dữ liệu (Polling API)	14
8. HỆ THỐNG BUILD VÀ TRIỂN KHAI.....	15
8.1. Toolchain biên dịch chéo (Cross-compilation).....	15
8.2. Các file cần triển khai lên board.....	15
9. LUỒNG HOẠT ĐỘNG TỔNG THỂ.....	16
9.1. Giai đoạn 1 — Khởi động hệ thống.....	16

9.2. Giai đoạn 2 — Vòng lặp chính	16
9.3. Giai đoạn 3 — Kết thúc	17
10. TÓM TẮT VÀ KẾT LUẬN.....	18
10.1. Tóm tắt kỹ thuật	18
10.2. Điểm nổi bật của thiết kế.....	18
10.3. Cấu hình thông số vận hành.....	19
11. Phân công công việc	20

1. TỔNG QUAN HỆ THỐNG

Hệ thống Đèn Thông Minh (Smart Lamp) là một giải pháp chiếu sáng tự động hoàn chỉnh, được phát triển và triển khai trên bo mạch BeagleBone Black Rev C tích hợp SoC AM335x (ARM Cortex-A8). Đây là một hệ thống nhúng (embedded system) đầy đủ, bao gồm cả phần cứng, driver kernel, ứng dụng userspace đa luồng và giao diện giám sát qua Web.

Hệ thống tích hợp 5 ngoại vi phần cứng giao tiếp trên 3 bus khác nhau (I2C, SPI, PWM), được điều khiển bởi 5 Linux Kernel Driver tùy chỉnh và một ứng dụng userspace viết bằng C với 4 luồng (thread) chạy song song.

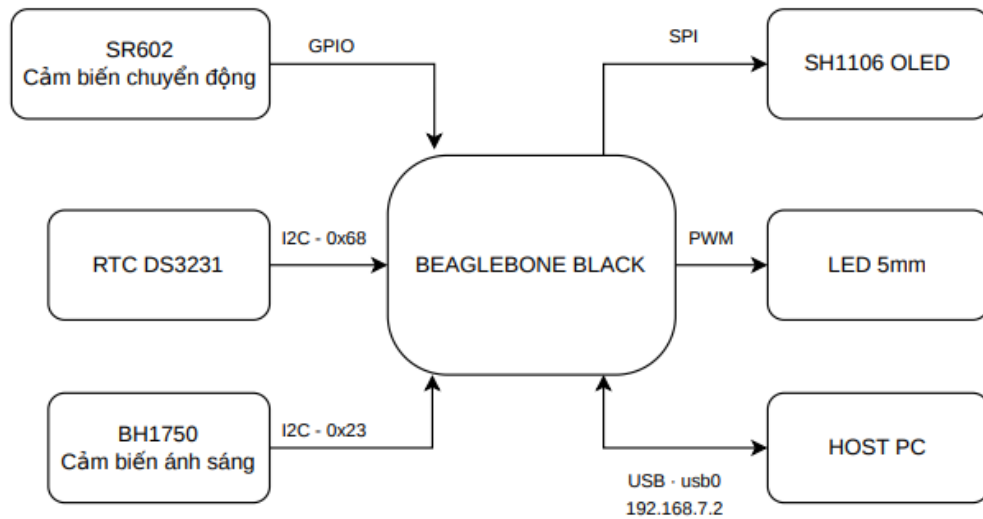
1.1. Mục tiêu thiết kế

Tự động bật/tắt đèn dựa trên thời gian thực (RTC), cường độ ánh sáng môi trường (lux), và phát hiện chuyển động (PIR). Điều khiển độ sáng mượt mà bằng PWM với hiệu ứng fade (chuyển tiếp từng bước $\pm 2\%$ /tick). Giám sát và điều khiển từ xa qua giao diện Web (HTTP server tự viết bằng C, port 8080). Hiển thị trạng thái hệ thống trực tiếp trên màn hình OLED 128×64. Khởi động tự động khi cấp nguồn thông qua init script Buildroot (plug-and-play). Ghi nhật ký tiêu thụ năng lượng theo giờ trong 24 giờ liên tục

1.2. Thành phần phần cứng

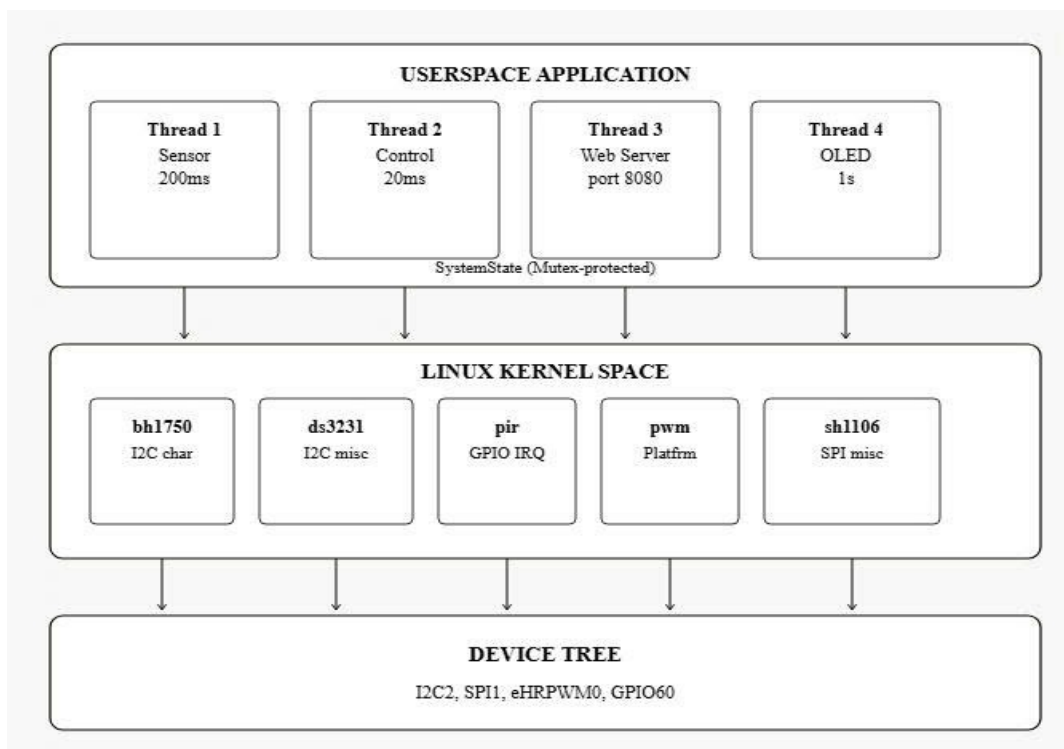
Hệ thống gồm 5 ngoại vi phần cứng kết nối với BeagleBone Black Rev C:

Thành phần	Chip/Module	Giao tiếp	Chân kết nối	Device Node
Cảm biến ánh sáng	BH1750	I2C2 (0x23)	P9_19, P9_20	/dev/bh1750
Đồng hồ thực (RTC)	DS3231	I2C2 (0x68)	P9_19, P9_20	/dev/myds3231
Cảm biến chuyển động	HC-SR501/SR602	GPIO Interrupt	P9_12 (GPIO60)	/dev/mypir
Điều khiển LED	LED 5mm	PWM eHRPWM0A	P9_22	sysfs attribute
Màn hình OLED	SH1106 128×64	SPI1 (8MHz)	P9_28–P9_31, P9_15, P9_23	/dev/myoled



1.3. Sơ đồ kiến trúc tổng thể

Kiến trúc phần mềm được tổ chức thành 3 tầng rõ ràng: Userspace Application (tầng ứng dụng), Linux Kernel Space (tầng driver), và Hardware (tầng phần cứng):



2. KIẾN TRÚC PHẦN MỀM

2.1. Cấu trúc dữ liệu dùng chung — SystemState

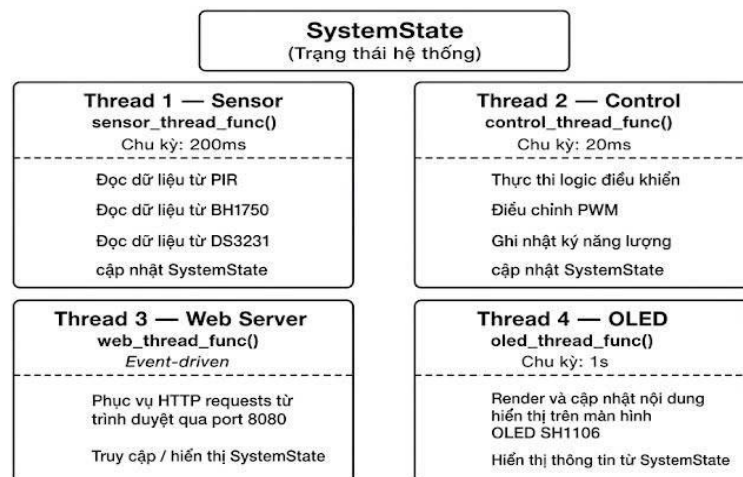
Toàn bộ 4 luồng trong hệ thống chia sẻ một cấu trúc dữ liệu trung tâm duy nhất có tên SystemState. Đây là trái tim của ứng dụng, chứa tất cả trạng thái hoạt động của hệ thống và được bảo vệ bởi pthread_mutex để đảm bảo an toàn đồng thời (thread-safe).

```
typedef struct {  
    int current_brightness; // Độ sáng hiện tại (0–100%)  
    int manual_target; // Giá trị đích do người dùng đặt qua Web  
    bool motion_detected; // Trạng thái cảm biến PIR  
    float ambient_lux; // Cường độ ánh sáng đo từ BH1750  
    bool manual_override; // Chế độ điều khiển thủ công  
    char current_time[32]; // Thời gian thực từ DS3231  
    bool is_dark_confirmed; // Xác nhận: trời tối (19h–6h AND lux<200)  
    bool is_night; // Trong khoảng thời gian ban đêm (19h–6h)  
    bool human_present; // Đã xác nhận có người hiện diện  
    int no_motion_ticks; // Đếm tick không có chuyển động  
    int auto_target; // Giá trị đích tự động tính toán  
    int energy_seconds[24]; // Thời gian đèn sáng (giây) theo từng giờ  
    int total_on_off_count; // Tổng số lần bật/tắt đèn  
    pthread_mutex_t lock; // Mutex bảo vệ toàn bộ cấu trúc  
} SystemState;
```

Mọi thao tác đọc/ghi trên SystemState đều phải thực hiện theo nguyên tắc: mutex_lock → thao tác → mutex_unlock. Điều này đảm bảo tính nhất quán dữ liệu khi 4 luồng truy cập đồng thời.

2.2. Mô hình đa luồng (Multi-threading)

Ứng dụng smart_lamp sử dụng POSIX threads (pthread) để tạo 4 luồng chạy song song ngay sau khi khởi tạo SystemState:



3. KERNEL DRIVERS

Hệ thống sử dụng 5 kernel module (.ko) tùy chỉnh, mỗi module quản lý một ngoại vi phần cứng. Tất cả được nạp theo thứ tự cụ thể bởi init script và tạo ra device nodes để ứng dụng userspace truy cập.

3.1. Driver BH1750 — Cảm biến ánh sáng môi trường

Thuộc tính	Giá trị
Loại driver	I2C Client Driver
Device node	/dev/bh1750
DTS compatible	rohm,bh1750
Địa chỉ I2C	0x23 trên bus I2C2
Lệnh đo	CMD_ONE_TIME_H_RES (0x20) — Độ phân giải cao, đo một lần
Thời gian đo	Chờ 180ms theo datasheet BH1750
Công thức Lux	(Byte_cao << 8 Byte_thấp) × 10 / 12 (tránh dùng float trong kernel)

Cơ chế hoạt động: Khi probe, driver đăng ký character device, tạo class và device node, đồng thời bật nguồn cảm biến bằng lệnh CMD_POWER_ON. Khi remove, driver gửi lệnh CMD_POWER_DOWN để tắt cảm biến và giải phóng device node. Ứng dụng userspace đọc giá trị lux bằng cách gọi read() trên /dev/bh1750.

3.2. Driver DS3231 — Đồng hồ thời gian thực (RTC)

Thuộc tính	Giá trị
Loại driver	I2C Client Driver (miscdevice)
Device node	/dev/myds3231
DTS compatible	my,ds3231
Địa chỉ I2C	0x68 trên bus I2C2
Số thanh ghi đọc/ghi	7 thanh ghi (địa chỉ 0x00–0x06)
Định dạng dữ liệu	BCD ↔ Decimal, chuỗi YYYY-MM-DD HH:MM:SS
Loại đăng ký	miscdevice với MISC_DYNAMIC_MINOR

Cơ chế hoạt động: Khi probe, driver kích hoạt oscillator bằng cách xóa bit ~EOSC trong Control Register (0x0E) và xóa cờ OSF trong Status Register (0x0F). Thao tác đọc (read) trả về chuỗi thời gian định dạng YYYY-MM-DD HH:MM:SS sau khi chuyển đổi BCD sang Decimal. Thao tác ghi (write) nhận chuỗi theo định dạng tương tự, chuyển Decimal sang BCD và ghi vào 7 thanh ghi.

3.3. Driver PIR — Cảm biến chuyển động

Thuộc tính	Giá trị
Loại driver	GPIO Interrupt Driver (character device thủ công)
Device node	/dev/mypir
Chân GPIO	GPIO60 — chân P9_12 trên BeagleBone Black
Cấu hình pull	Pull-down nội: devmem 0x44E10878 32 0x27
Loại ngắt	Cả 2 cạnh: IRQF_TRIGGER_RISING IRQF_TRIGGER_FALLING
Kết quả đọc	Trả '1' (có chuyển động) hoặc '0' (không có) + newline
Quản lý tài nguyên	gpio_request → gpio_direction_input → gpio_to_irq → request_irq

Cơ chế hoạt động: ISR (Interrupt Service Routine) được kích hoạt trên cả cạnh lên và cạnh xuống của tín hiệu GPIO, cập nhật ngay biến motion_detected trong kernel. Ứng dụng userspace đọc trạng thái này qua read() trên /dev/mypir. Cơ chế debounce được thực hiện ở tầng userspace (Thread 2 — Control) thay vì ở driver.

3.4. Driver PWM — Điều khiển LED

Thuộc tính	Giá trị
Loại driver	Platform Driver
Giao diện	sysfs: /sys/class/smart_lamp/led/brightness
DTS compatible	son,smart-led
Nguồn PWM	eHRPWM0, channel 0, chân P9_22
Tần số hoạt động	1 kHz (chu kỳ = 1.000.000 ns)
Phạm vi điều chỉnh	0–100 (ánh xạ tuyến tính sang duty cycle)
Công thức duty cycle	duty_ns = (val × PERIOD_NS) / 100

Cơ chế hoạt động: Khi probe, driver sử dụng devm_pwm_get() để lấy thiết bị PWM từ Device Tree và khởi tạo ở trạng thái tắt (duty cycle = 0). Thuộc tính sysfs DEVICE_ATTR_RW(brightness) cung cấp giao diện đọc/ghi giá trị 0–100. Thread 2 (Control) ghi độ sáng mong muốn vào sysfs attribute này sau khi tính toán hiệu ứng fade.

3.5. Driver SH1106 — Màn hình OLED 128×64

Thuộc tính	Giá trị
Loại driver	SPI Driver (miscdevice)
Device node	/dev/myoled
DTS compatible	my,sh1106-spi
Giao tiếp SPI	8 bit/word, tốc độ tối đa 8 MHz
Chân điều khiển	DC: P9_15 (gpio1_16), Reset: P9_23 (gpio1_17)
Chuỗi khởi tạo	18 lệnh cấu hình: Display OFF → Clock → Multiplex → Contrast → Display ON
Kích thước framebuffer	1024 bytes (8 trang × 128 bytes/trang)

Cơ chế hoạt động: Ứng dụng userspace chuẩn bị toàn bộ framebuffer 1024 bytes trong RAM, sau đó ghi một lần vào /dev/myoled. Driver nhận dữ liệu và ghi theo từng trang (8 trang, mỗi trang 128 bytes), áp dụng offset cột 2 pixel đặc trưng của chip SH1106.

4. ỨNG DỤNG USERSPACE

Ứng dụng chính smart_lamp (file main.c, 481 dòng) là trung tâm điều phối toàn bộ hệ thống. Sau khi khởi tạo SystemState và pthread_mutex, ứng dụng tạo 4 luồng chạy song song và bắt đầu vòng lặp hoạt động chính.

4.1. Thread 1 — Đọc cảm biến

Thread 1 chịu trách nhiệm thu thập dữ liệu từ tất cả cảm biến phần cứng. Mỗi 200ms, luồng thực hiện tuần tự:

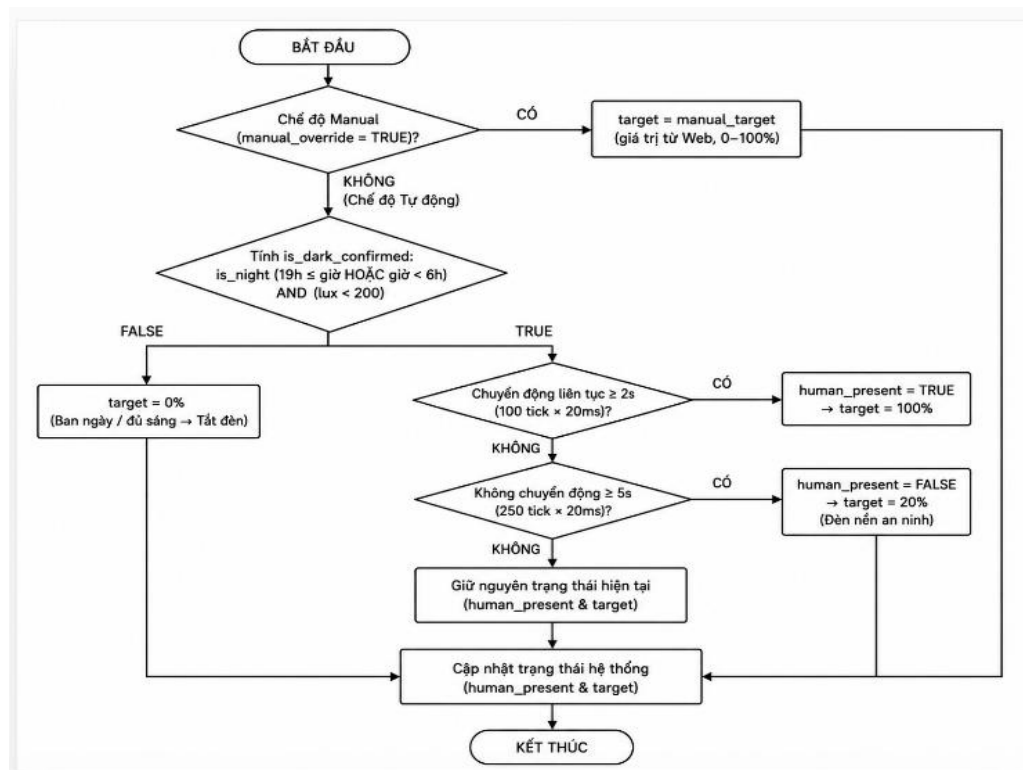
- Đọc trạng thái PIR từ /dev/mypir → cập nhật motion_detected
- Đọc cường độ ánh sáng từ /dev/bh1750 → cập nhật ambient_lux
- Đọc thời gian thực từ /dev/myds3231 → cập nhật current_time
- Khóa mutex → ghi tất cả giá trị vào SystemState → mở mutex

Chu kỳ 200ms là sự cân bằng giữa độ tươi của dữ liệu và chi phí đọc I2C/GPIO. Thread này không thực hiện bất kỳ logic điều khiển nào — chỉ đơn thuần thu thập và lưu trữ dữ liệu thô.

4.2. Thread 2 — Logic điều khiển (Control Thread, chu kỳ 20ms)

Thread 2 là luồng quan trọng nhất, thực thi thuật toán điều khiển mỗi 20ms (50 lần/giây). Chu kỳ ngắn này đảm bảo hiệu ứng fade PWM hoạt động mượt mà.

4.2.1. Thuật toán quyết định độ sáng mục tiêu



4.2.2. Hiệu ứng Fade PWM

Thay vì bật/tắt đột ngột, Thread 2 điều chỉnh độ sáng từng bước ± 2 đơn vị mỗi chu kỳ 20ms tiến về giá trị target. Điều này tạo ra chuyển tiếp mềm mại trong khoảng 1 giây ($50 \text{ bước} \times 2\% = 100\%$). Giá trị được ghi trực tiếp vào sysfs attribute của PWM driver.

4.2.3. Ghi nhật ký năng lượng

Mỗi 50 tick (tương đương 1 giây thực) khi đèn đang sáng, Thread 2 cộng dồn 1 vào `energy_seconds[hour]` tương ứng với giờ hiện tại. Mảng 24 phần tử này lưu tổng số giây đèn sáng trong mỗi giờ trong ngày. Đồng thời, biến `total_on_off_count` được tăng mỗi khi đèn chuyển từ trạng thái TẮT sang BẬT.

4.3. Thread 3 — Web Server

Thread 3 triển khai một HTTP server tối giản bằng C thuần (raw TCP socket), không sử dụng bất kỳ framework hay thư viện bên ngoài nào. File `index.html` được đọc vào RAM khi khởi động để tối ưu tốc độ phục vụ.

Cấu hình socket: `SO_REUSEADDR` (tái sử dụng địa chỉ khi khởi động lại) và `SO_RCVTIMEO = 1` giây (timeout tự động đóng connection bị treo, tránh deadlock). Server chấp nhận kết nối bằng `accept()`, đọc HTTP request, phân tích method và endpoint, sau đó ghi response và đóng kết nối.

4.4. Thread 4 — Hiển thị OLED (chu kỳ 1 giây)

Thread 4 cập nhật màn hình OLED mỗi 1 giây. Luồng đọc trạng thái từ `SystemState`, sau đó render toàn bộ nội dung thành framebuffer 1024 bytes sử dụng font bitmap 5×7 pixel (định nghĩa trong `font.h`).

Sau khi render xong, toàn bộ 1024 bytes framebuffer được ghi một lần vào `/dev/myoled`. Driver SH1106 tiếp nhận và ghi lên màn hình theo từng trang.

5. DEVICE TREE (DTS)

Device Tree Source (DTS) là cơ chế mô tả phần cứng trong Linux kernel hiện đại. File `am335x-boneblack.dts` khai báo toàn bộ phần cứng của hệ thống, bao gồm cấu hình pin mux và các node thiết bị.

5.1. Cấu hình Pin Mux

Ngoại vi	Chân vật lý	Mode	Cấu hình thanh ghi
I2C2 (SCL)	P9_19	MODE3	SLEWCTRL_SLOW + INPUT_PULLUP (0x73)
I2C2 (SDA)	P9_20	MODE3	SLEWCTRL_SLOW + INPUT_PULLUP (0x73)
SPI1 CS0	P9_28	MODE3	INPUT_PULLUP (0x33)
SPI1 D0 (MISO)	P9_29	MODE3	INPUT_PULLUP (0x33)
SPI1 D1 (MOSI)	P9_30	MODE3	INPUT_PULLUP (0x33)
SPI1 SCLK	P9_31	MODE3	INPUT_PULLUP (0x33)
eHRPWM0A (LED)	P9_22	MODE3	Output (0x03)

5.2. Node thiết bị

- I2C2 (clock-frequency = 100kHz): `my_bh1750@23` (compatible `rohm,bh1750`), `my_ds3231@68` (compatible `my,ds3231`)
- SPI1: `my_sh1106_spi@0` (compatible `my,sh1106-spi`, max-frequency 8MHz, `dc-gpios` = `gpio1_16`, `reset-gpios` = `gpio1_17`)
- PWM: `my_smart_led` (compatible `son,smart-led`, `pwms` = `ehrpwm0` channel 0, `period-ns` = 1.000.000)
- `mcpasp0`: `status` = `disabled` — vô hiệu hóa vì xung đột chân với SPI1

6. QUY TRÌNH KHỞI ĐỘNG VÀ DỪNG HỆ THỐNG

6.1. Trình tự khởi động tự động — S99smartlamp

Khi cấp nguồn, Buildroot Linux thực thi init script S99smartlamp từ /etc/init.d/ theo trình tự được kiểm soát chặt chẽ:

Bước	Hành động	Thời gian chờ	Lý do
1	Chờ hệ thống ổn định	15 giây	Đảm bảo điện áp nguồn và I2C bus đã ổn định hoàn toàn sau khi cấp nguồn
2	Cấu hình GPIO P9_12	—	Lệnh devmem 0x44E10878 32 0x27 bật pull-down nội cho PIR, loại bỏ nhiễu tín hiệu
3	Nạp 5 kernel driver	—	insmod theo thứ tự: pwm → pir → bh1750 → ds3231 → sh1106_spi
4	Chờ device nodes	2 giây	Đảm bảo kernel đã tạo đầy đủ /dev/bh1750, /dev/mypir, v.v. trước khi app chạy
5	Khởi chạy ứng dụng	—	Thực thi ./smart_lamp, chuyển hướng log vào /var/log/smart_lamp.log

6.2. Trình tự dừng hệ thống (SIGINT/SIGTERM)

- Lệnh killall smart_lamp gửi tín hiệu SIGTERM đến tiến trình
- 4 luồng nhận tín hiệu và thoát khỏi vòng lặp while(1) một cách có kiểm soát
- Gọi rmmod theo thứ tự ngược: sh1106 → ds3231 → bh1750 → pir → pwm
- pthread_mutex_destroy giải phóng tài nguyên đồng bộ hóa
- Hệ thống dừng hoàn toàn — tất cả device nodes bị xóa

Thứ tự ngược khi rmmod đảm bảo không có driver nào bị xóa khi còn driver khác phụ thuộc vào nó, tránh lỗi kernel panic hoặc rò rỉ tài nguyên.

7. GIAO DIỆN WEB DASHBOARD

7.1. Các thành phần giao diện

Thành phần	Chức năng chi tiết
Header	Hiển thị tên hệ thống, thời gian cập nhật lần cuối, badge trạng thái kết nối (online/offline)
Quick Stats (4 ô)	Ánh sáng môi trường (lux), Trạng thái chuyển động (có/không người), Thời gian RTC, Độ sáng đèn hiện tại (%)
Trạng thái hệ thống	Chế độ hoạt động (Tự động/Thủ công), is_dark, is_night, lux < ngưỡng, motion_confirmed, timeout đếm ngược, quyết định và lý do
Điều khiển thủ công	Toggle bật/tắt chế độ manual override, slider điều chỉnh độ sáng 0–100%, nút Áp dụng
Thống kê năng lượng	Tổng số lần bật đèn, tổng giờ sáng tích lũy, biểu đồ cột 24 cột hiển thị thời gian sáng theo từng giờ
Nhật ký hoạt động	Log sự kiện bật/tắt theo thời gian thực, phân biệt chế độ đêm/ngày, giữ tối đa 50 entry gần nhất

7.2. Cơ chế cập nhật dữ liệu (Polling API)

API Endpoint	Tần suất polling	Dữ liệu nhận được
/api/status	Mỗi 500ms	brightness, motion, lux, manual, time, is_dark, is_night, human_present, no_motion_ticks, auto_target
/api/energy	Mỗi 2000ms	on_off_count, hours[24] — mảng 24 giá trị giây sáng theo từng giờ

Giao diện sử dụng JavaScript để gọi fetch() theo chu kỳ và cập nhật DOM mà không cần reload trang. Tần suất 500ms cho /api/status đảm bảo hiển thị gần real-time mà không tạo quá tải cho Web server C.

8. HỆ THỐNG BUILD VÀ TRIỂN KHAI

8.1. Toolchain biên dịch chéo (Cross-compilation)

Toàn bộ mã nguồn được biên dịch chéo trên máy host Linux cho kiến trúc ARM hard-float:

Thành phần	Công cụ/Chi tiết
Compiler	arm-buildroot-linux-gnueabi-hf-gcc (từ Buildroot toolchain)
CFLAGS ứng dụng	-Wall -Wextra -pthread (kèm cờ cảnh báo đầy đủ và hỗ trợ POSIX thread)
Build ứng dụng	src/main.c → binary smart_lamp
Build kernel modules	5 file .c → 5 file .ko (pwm, pir, bh1750, ds3231, sh1106_spi)
KDIR	Trở đến output/build/linux-custom của Buildroot
Architecture	ARCH=arm CROSS_COMPILE=arm-buildroot-linux-gnueabi-hf-

8.2. Các file cần triển khai lên board

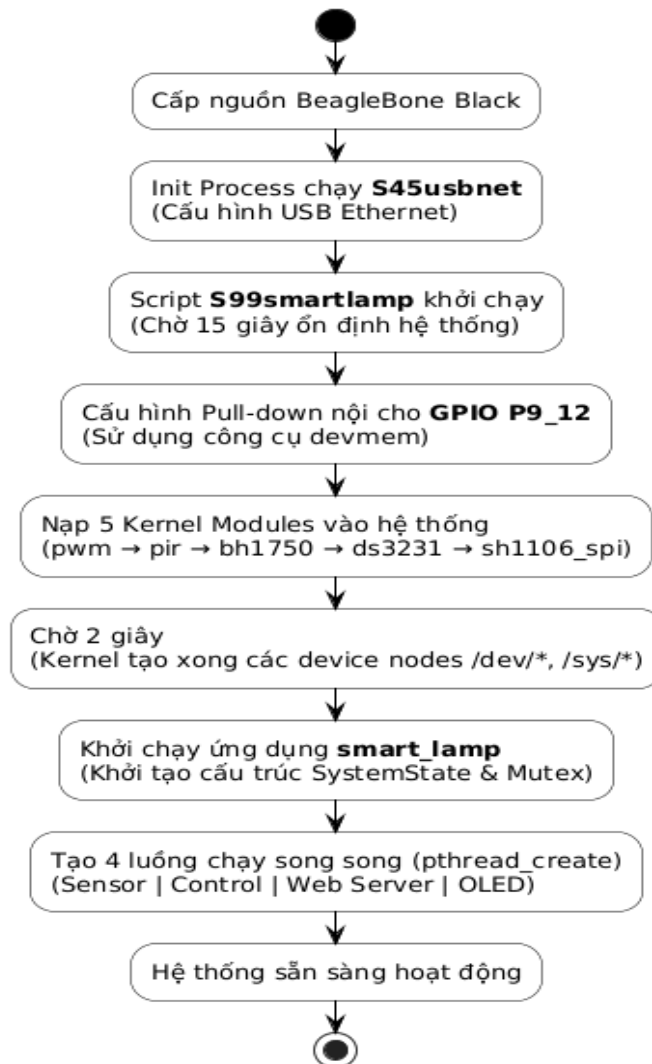
Tất cả file cần được copy vào thư mục /root/ trên BeagleBone Black qua SCP:

File	Loại	Mô tả
smart_lamp	Binary ELF (ARM)	Ứng dụng userspace chính đã biên dịch chéo
pwm_driver.ko	Kernel module	Driver điều khiển LED PWM eHRPWM0A
pir_driver.ko	Kernel module	Driver cảm biến chuyển động GPIO IRQ
bh1750_driver.ko	Kernel module	Driver cảm biến ánh sáng I2C
ds3231_driver.ko	Kernel module	Driver đồng hồ thực (RTC) I2C
sh1106_spi_driver.ko	Kernel module	Driver màn hình OLED SPI
www/index.html	HTML/CSS/JS	Giao diện Web dashboard (413 dòng)
am335x-boneblack.dtb	Device Tree Blob	Mô tả phần cứng đã biên dịch (từ DTS)

Init scripts S45usbnet và S99smartlamp cần được copy vào /etc/init.d/ và cấp quyền thực thi (chmod +x). S45usbnet cấu hình USB Ethernet (usb0, IP 192.168.7.2/24) để kết nối SSH và SCP từ máy host.

9. LƯỒNG HOẠT ĐỘNG TỔNG THỂ

9.1. Giai đoạn 1 — Khởi động hệ thống



9.2. Giai đoạn 2 — Vòng lặp chính

Sau khi khởi động, hệ thống vận hành liên tục trong 4 vòng lặp song song, kết nối với nhau qua cấu trúc SystemState được bảo vệ mutex:

Luồng	Vòng lặp	Hành động mỗi chu kỳ
Thread 1 — Sensor	200ms/vòng	Đọc /dev/mypir, /dev/bh1750, /dev/myds3231 → mutex_lock → ghi SystemState → mutex_unlock → usleep 200ms
Thread 2 — Control	20ms/vòng	mutex_lock → đọc state → mutex_unlock → tính is_dark → quyết định target → fade PWM → energy_log → usleep 20ms
Thread 3 — Web	Event- driven	accept() → đọc HTTP request (timeout 1s) → parse method & URL → route API → đọc/ghi SystemState → gửi response → close
Thread 4 — OLED	1s/vòng	mutex_lock → đọc state → mutex_unlock → render framebuffer 1024B → write /dev/myoled → usleep 1s

9.3. Giai đoạn 3 — Kết thúc

- Nhận SIGINT (Ctrl+C) hoặc SIGTERM (từ init stop): cờ dừng được thiết lập
- 4 luồng phát hiện cờ dừng và thoát khỏi while(1) theo cách có kiểm soát
- pthread_join chờ các luồng kết thúc sạch
- rmmmod 5 driver theo thứ tự ngược: sh1106 → ds3231 → bh1750 → pir → pwm
- pthread_mutex_destroy giải phóng mutex → Hệ thống dừng hoàn toàn

10. TÓM TẮT VÀ KẾT LUẬN

10.1. Tóm tắt kỹ thuật

Hệ thống Đèn Thông Minh là một giải pháp embedded hoàn chỉnh từ phần cứng đến phần mềm, được thiết kế và triển khai với kiến trúc phân tầng rõ ràng:

Tầng	Thành phần	Số lượng/Quy mô
Phần cứng	Ngoại vi kết nối trên 3 bus: I2C2, SPI1, GPIO, PWM	5 thiết bị
Kernel Space	Linux Kernel Drivers tùy chỉnh (I2C/SPI/GPIO/PWM)	5 module .ko
Userspace App	Ứng dụng đa luồng C với POSIX pthread và mutex	4 threads, 481 dòng C
Giao diện Web	HTTP server C thuần + Dashboard HTML/CSS/JS	4 API endpoints, 413 dòng HTML
Hệ thống Build	Buildroot Linux cross-compilation toolchain (ARM hard-float)	Toolchain đầy đủ
Init System	Buildroot init scripts khởi động/dừng tự động	2 scripts
Device Tree	DTS mô tả đầy đủ phần cứng, pin mux, và node thiết bị	1 file DTS

10.2. Điểm nổi bật của thiết kế

- Kiến trúc phân tầng rõ ràng: Hardware → Kernel Driver → SystemState → Userspace Threads → Web Interface, mỗi tầng độc lập và có giao diện xác định
- Đồng bộ hóa an toàn: Tất cả truy cập chia sẻ đều thông qua pthread_mutex, đảm bảo thread-safety tuyệt đối trong môi trường 4 luồng đồng thời
- Điều khiển phản hồi thông minh: Kết hợp 3 yếu tố (thời gian, ánh sáng, chuyển động) với debounce phần mềm tránh bật/tắt nhấp nháy khi người đi lại
- Hiệu ứng fade mượt mà: Chuyển tiếp độ sáng $\pm 2\%/20\text{ms}$ tạo trải nghiệm thị giác dễ chịu, khác biệt với bật/tắt đột ngột
- Giám sát năng lượng: Ghi nhật ký theo từng giờ trong 24 giờ cho phép phân tích xu hướng sử dụng điện
- Plug-and-play: Toàn bộ hệ thống khởi động tự động khi cấp nguồn, không cần can thiệp thủ công
- Web server tự triển khai: Không phụ thuộc framework, giảm thiểu tài nguyên hệ thống — phù hợp với môi trường embedded

10.3. Cấu hình thông số vận hành

Thông số	Giá trị	Ghi chú
Ngưỡng ánh sáng	200 lux	Dưới ngưỡng này, kết hợp với is_night → kích hoạt chế độ tối
Khung giờ ban đêm	19:00 – 6:00	is_night = TRUE trong khoảng thời gian này
Thời gian xác nhận chuyển động	2 giây (100 tick × 20ms)	Tránh báo nhầm do nhiễu PIR
Thời gian timeout không người	5 giây (250 tick × 20ms)	Sau đó chuyển về đèn nền 20%
Độ sáng tối đa (có người)	100%	PWM duty cycle 100%
Độ sáng nền an ninh	20%	Đèn nền khi trời tối nhưng không có người
Tốc độ fade	±2% mỗi 20ms	Chuyển tiếp hoàn chỉnh trong ~1 giây
Chu kỳ đọc cảm biến	200ms	Tần suất cập nhật lux, PIR, thời gian
Tần số PWM LED	1 kHz	Không gây nhấp nháy nhìn thấy được

11. Phân công công việc

Thành viên	Nội dung công việc
Nguyễn Hữu Sơn	- Viết driver PIR (GPIO interrupt) - Viết driver PWM điều khiển LED - Xây dựng Web Server (TCP socket) - Tạo API nội bộ - Thiết kế Web UI (HTML/CSS/JS, biểu đồ, slider)
Lương Công Thắng	- Viết driver OLED SH1106 (SPI) - Xử lý framebuffer hiển thị - Xây dựng OLED Thread (1s) - Render dữ liệu lên màn hình - Build system (Makefile, Buildroot, script khởi động)
Đỗ Đức Thuận	- Viết driver DS3231 (RTC - I2C) - Xử lý BCD ↔ Decimal - Xây dựng Control Thread (20ms) - Logic bật/tắt đèn theo thời gian & ánh sáng - Energy logging, hiệu ứng fade
Trịnh Quang Huy	- Cấu hình Device Tree (DTS) - Viết driver BH1750 (I2C) - Xử lý đọc Lux - Xây dựng Sensor Thread (đọc dữ liệu 200ms)